

Workflow Engine Scaffold

Workflow Engine Scaffold

Overview

This document provides a complete workflow engine scaffold for the Agent Dashboard MVP. It includes folder structure, TypeScript starter files, RxJS-based graph execution, node/edge models, and workflow registry. This scaffold is designed to align with the /spec contracts and integrate seamlessly with the backend and frontend scaffolds.

Folder Structure

```
/workflow
/engine
  workflowEngine.ts
  nodeExecutor.ts
  edgeExecutor.ts
/models
  WorkflowNode.ts
  WorkflowEdge.ts
  WorkflowDefinition.ts
/services
  WorkflowRegistry.ts
/canvas
  WorkflowCanvasPlaceholder.tsx
```

File: /models/WorkflowNode.ts

```
export type WorkflowNodeType = "agent" | "transform" | "trigger" | "branch";

export interface WorkflowNode {
  id: string;
  type: WorkflowNodeType;
  sessionId: string | null;
  config: Record<string, any>;
  position: { x: number; y: number };
  inputs: string[];
  outputs: string[];
}
```

File: /models/WorkflowEdge.ts

```
export type WorkflowOperator = "map" | "merge" | "switchMap" | "combineLatest" | "filter"  
| null;  
  
export interface WorkflowEdge {  
  id: string;  
  fromNodeId: string;  
  fromPort: string;  
  toNodeId: string;  
  toPort: string;  
  operator: WorkflowOperator;  
}
```

File: /models/WorkflowDefinition.ts

```
import { WorkflowNode } from "../WorkflowNode";  
import { WorkflowEdge } from "../WorkflowEdge";  
  
export interface WorkflowDefinition {  
  id: string;  
  name: string;  
  nodes: WorkflowNode[];  
  edges: WorkflowEdge[];  
}
```

File: /services/WorkflowRegistry.ts

```
import { WorkflowDefinition } from "../models/WorkflowDefinition";

export const WorkflowRegistry = {
  workflows: new Map<string, WorkflowDefinition>(),

  register(def: WorkflowDefinition) {
    this.workflows.set(def.id, def);
  },

  get(id: string): WorkflowDefinition | undefined {
    return this.workflows.get(id);
  },

  list(): WorkflowDefinition[] {
    return Array.from(this.workflows.values());
  }
};
```

File: /engine/nodeExecutor.ts

```
import { sessionEvents } from "../../frontend/state/eventBus";
import { WorkflowNode } from "../models/WorkflowNode";
import { Observable } from "rxjs";
import { filter } from "rxjs/operators";

export function executeNode(node: WorkflowNode): Observable<any> {
  if (!node.sessionId) throw new Error("Node has no sessionId");

  return sessionEvents(node.sessionId).pipe(
    filter(e => e.type === "output.final" || e.type === "output.partial")
  );
}
```

File: /engine/edgeExecutor.ts

```
import { Observable } from "rxjs";
import { map, merge, switchMap, combineLatest, filter as rxFilter } from "rxjs/operators";
import { WorkflowEdge } from "../models/WorkflowEdge";

export function applyEdgeOperator(edge: WorkflowEdge, streams: Record<string,
Observable<any>>): Observable<any> {
  const source = streams[edge.fromNodeId];

  switch (edge.operator) {
    case "map":
      return source.pipe(map(v => ({ ...v, mapped: true })));

    case "merge":
      return merge(source);

    case "switchMap":
      return source.pipe(switchMap(v => streams[edge.toNodeId]));

    case "combineLatest":
      return combineLatest([source, streams[edge.toNodeId]]);

    case "filter":
      return source.pipe(rxFilter(v => !!v));

    default:
      return source;
  }
}
```

 File: /engine/workflowEngine.ts

```

import { WorkflowDefinition } from "../models/WorkflowDefinition";
import { executeNode } from "../nodeExecutor";
import { applyEdgeOperator } from "../edgeExecutor";
import { Observable } from "rxjs";

export class WorkflowEngine {
  private definition: WorkflowDefinition | null = null;
  private streams: Record<string, Observable<any>> = {};

  loadWorkflow(def: WorkflowDefinition) {
    this.definition = def;
    this.initializeStreams();
  }

  initializeStreams() {
    if (!this.definition) return;

    this.streams = {};

    for (const node of this.definition.nodes) {
      this.streams[node.id] = executeNode(node);
    }

    for (const edge of this.definition.edges) {
      this.streams[edge.toNodeId] = applyEdgeOperator(edge, this.streams);
    }
  }

  subscribe(nodeId: string, callback: (event: any) => void) {
    const stream = this.streams[nodeId];
    if (!stream) throw new Error(`No stream for node ${nodeId}`);
    return stream.subscribe(callback);
  }

  start() {
    console.log("Workflow started");
  }

  stop() {
    console.log("Workflow stopped");
  }
}

```

File: /canvas/WorkflowCanvasPlaceholder.tsx

```
import React from "react";

export function WorkflowCanvasPlaceholder() {
  return (
    <div className="workflow-canvas-placeholder">
      <h2>Workflow Builder Coming Soon</h2>
      <p>This placeholder ensures the workflow engine integrates without future refactors.</p>
    </div>
  );
}
```

Summary

This workflow engine scaffold provides:

- Full workflow engine structure
- Node and edge models
- RxJS-based graph execution
- Workflow registry
- Canvas placeholder for future UI

It is fully aligned with the /spec contracts and ready for integration.